Graph

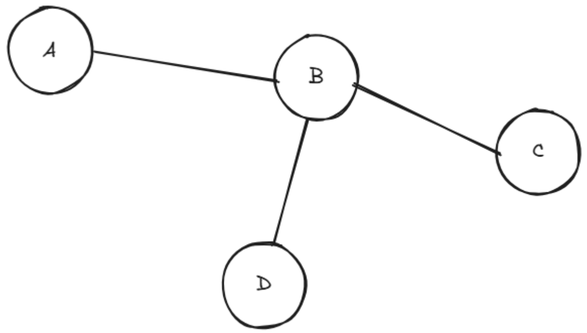
2026-1학기 부경대학교 알고리즘 스터디

목차

- 그래프(Graph)의 기초
- 그래프 순회 (Graph Traversal)
- 깊이 우선 탐색 (Depth-First Search)
- 너비 우선 탐색 (Breadth-First Search)

그래프(Graph)의 정의

- 수학적 정의, $G = (V, E)$
  - 그래프 G는 정점들의 집합 V와 이들을 잇는 간선들의 집합 E의 순서쌍입니다.
- 개념적 정의
  - 객체(Object)들 사이의 연결 관계를 점과 선으로 표현한 비선형 자료구조입니다.

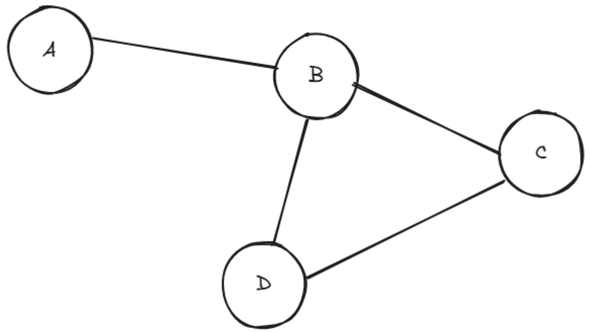


그래프의 구성 요소

- 정점 (Node)
  - 데이터를 저장하는 기본 단위입니다.
  - 현실 세계의 객체를 의미합니다. (예: 도시, 사람, 컴퓨터)
- 간선 (Edge)
  - 정점과 정점 사이를 잇는 선입니다.
  - 정점 간의 관계를 의미합니다. (예: 도로, 친구 관계, 통신 선로)

그래프 기초

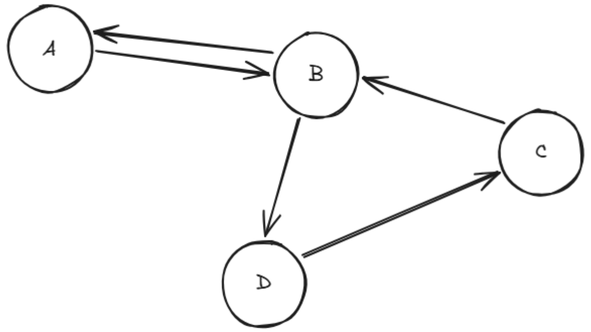
- 무방향 그래프 (Undirected Graph)
  - 대칭성: 정점 A와 B가 연결되어 있다면, A에서 B로 B에서 A로 이동할 수 있습니다.
  - 수학적 표현: 간선을 집합 {u, v}로 표시합니다. (집합은 순서가 상관없으므로 {u, v} = {v, u} 입니다.)
  - 실생활 비유: 친구 관계



그래프 기초

- 방향 그래프 (Directed Graph)
  - 비대칭성: A에서 B로 가는 간선이 있다고 해서 B에서 A로 갈 수 있는 것은 아닙니다.
  - 수학적 표현: 간선을 순서쌍 (u, v) 로 표시합니다. (순서쌍은 순서가 중요해서 (u, v) != (v, u) 입니다.)
  - 실생활 비유: 인스타그램 팔로우, 팔로워

    ※ 내가 장원영을 팔로우한다고 해서, 장원영이 나를 맞팔로우하는 것은 아님



그래프 기초

- 차수 (Degree)
  - 정점에 연결된 간선의 수를 의미하며, 해당 정점이 얼마나 많은 관계를 맺고 있는지 나타냅니다.
- 무방향 그래프에서의 차수
  - 단순히 연결된 선의 개수
- 방향 그래프에서의 차수
  - 진입차수 (In-degree): 해당 정점으로 들어오는 화살표의 수

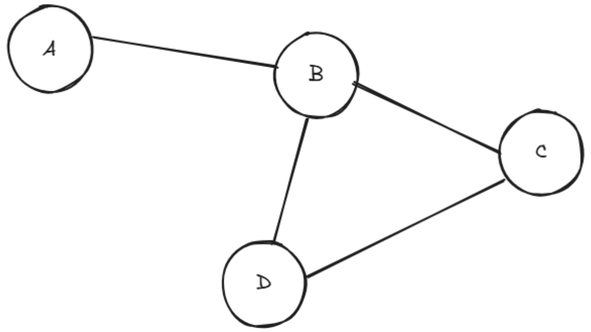
    ※ (예: 나를 팔로우하는 사람 수)
  - 진출차수 (Out-degree): 해당 정점에서 나가는 화살표의 수

    ※ (예: 내가 팔로잉하는 사람 수)

그래프 기초

- 비가중치 그래프 (Unweighted Graph)
  - 모든 간선의 '가치'나 '거리'가 동일하다고 가정하는 그래프입니다.
  - 연결성 중심: 두 정점이 이어져 있는가(0 또는 1)가 가장 중요합니다.
  - 거리의 정의: 두 정점 사이를 이동할 때 거쳐야 하는 간선의 최소 개수를 의미합니다.

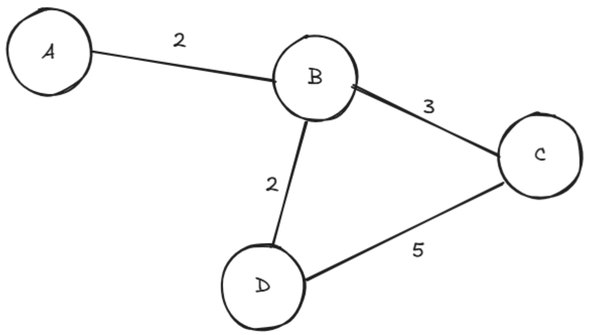
    ※ A에서 C까지 A -> B -> C, 거리는 2



그래프 기초

- 가중치 그래프 (Weighted Graph)
  - 각 간선마다 가중치(비용, 거리, 시간 등)가 할당된 그래프입니다.
  - 수학적 표현: 간선 (u, v)에 대해 가중치 함수 w(u, v) = W 로 표현합니다.
  - 거리의 정의: 두 정점을 연결하는 경로를 구성하는 간선들의 가중치 합이 최소가 되는 값을 의미합니다.

    ※ A에서 C까지 A -> B -> C, 거리는 2 + 3 = 5



그래프 기초

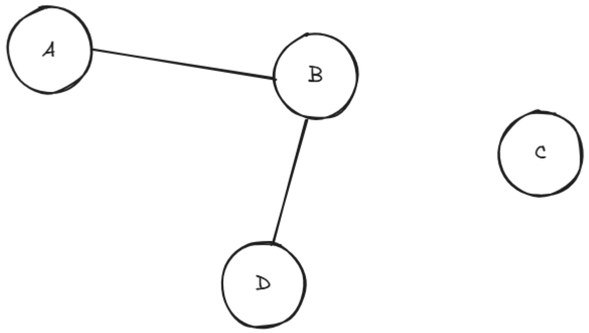
- 경로 (Path)
  - 정점 u에서 v까지 간선을 따라 이동할 수 있는 정점들의 나열입니다.
  - 길이 (Length): 경로를 이루는 간선(Edge)의 개수입니다.
    ※ 단, 가중치 그래프의 경우 간선 가중치의 총합을 의미
  - 단순 경로 (Simple Path): 경로 상에 중복된 정점이 없는 경우를 말합니다.
  - 두 정점 사이에는 이동 순서에 따라 서로 다른 여러 개의 경로가 존재할 수 있습니다.
- 사이클 (Cycle)
  - 시작 정점과 끝 정점이 같은 경로입니다. (다시 자기 자신으로 돌아오는 경로)
  - 단순 사이클 (Simple Cycle): 시작점과 끝점을 제외하고는 경로 내에 중복된 정점이 없는 사이클입니다.
  - 그래프 탐색 시 사이클을 적절히 처리하지 않으면, 컴퓨터는 무한 루프에 빠져 영원히 탐색을 끝내지 못합니다.

그래프의 종류

- 단순 그래프 (Simple Graph)
  - 정의: 루프(Self-loop)와 중복 간선(Multi-edge)이 없는 그래프입니다.

    ※ 루프: 정점 자기 자신으로 돌아오는 간선
    ※ 중복 간선: 두 정점 사이에 선이 여러 개인 경우

  - 가장 정제된 형태의 그래프로, 대부분의 알고리즘 문제에서 가정하는 기본 모델입니다.

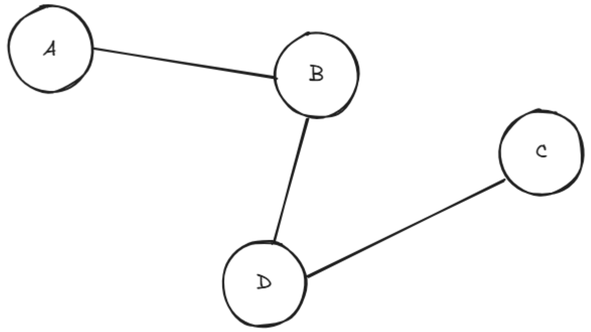


그래프의 종류

- 연결 그래프 (Connected Graph)
  - 정의: 그래프 내의 모든 정점 쌍 사이에 경로가 존재하는 형태입니다.

    ※ 무방향 그래프 기준
    ※ 고립된 정점이 없는 상태

  - 연결 요소(Connected Component): 비연결 그래프에서 서로 연결된 정점들끼리 묶인 개별적인 덩어리들을 의미합니다.

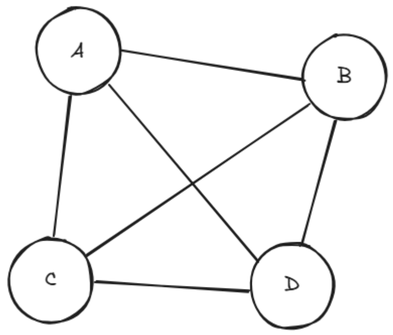


그래프의 종류

- 완전 그래프 (Complete Graph)
  - 정의: 모든 정점이 서로 직접적인 간선으로 연결된 그래프입니다.
  - 최대 간선 수: 정점이 V개일 때, 간선의 개수는 V x (V-1) /2 개입니다. (무방향 기준)

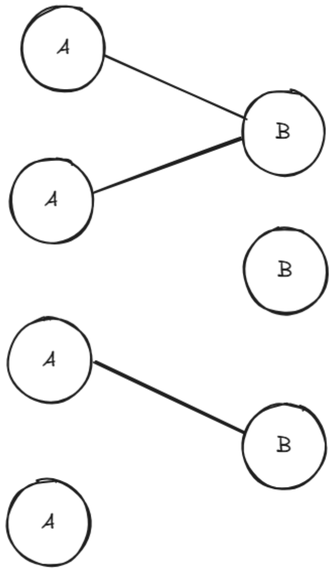
    ※ 무방향 그래프 기준

    ※ 정점이 조금만 많아져도 간선 수가 폭발적으로 늘어남



그래프의 종류

- 이분 그래프 (Bipartite Graph)
  - 정의: 정점들을 두 개의 집합 A, B로 나누었을 때, 같은 집합에 속한 정점끼리는 간선이 존재하지 않는 그래프입니다.
  - 학생-수강과목 같은 서로 다른 성격의 집합을 매칭하는 문제(네트워크 플로우 등)의 기초가 됩니다.



그래프의 표현 방법

- 인접 행렬 (Adjacency Matrix)
  - V x V 크기의 2차원 배열을 사용하여 정점 간의 연결 관계를 표시합니다.
  - adj[u][v] 값이 1(또는 가중치)이면 연결, 0이면 비연결을 의미합니다.
  - 연결 확인 O(1): 두 정점 u, v가 이어져 있는지 즉시 확인할 수 있습니다.
  - 공간 복잡도 $O(V^2)$: 정점 수의 제곱에 비례하는 메모리가 필요합니다.

그래프의 표현 방법

- 인접 행렬 (Adjacency Matrix)

```
// V x V 크기의 행렬을 0으로 초기화하여 생성한다.
vector<vector<int>> adj(V + 1, vector<int>(V + 1, 0));

for (int i = 0; i < E; i++) {
    int u, v;
    cin >> u >> v;
    adj[u][v] = 1;
    adj[v][u] = 1; // 무방향 그래프일 경우 대칭으로 저장한다.
}
```

그래프의 표현 방법

- 인접 리스트 (Adjacency List)
  - 각 정점마다 연결된 이웃 노드들만 리스트(C++에서는 vector)에 저장합니다.
  - vector<int> adj[V] 형태로, 각 정점마다 가변 배열을 가집니다.
  - 연결 확인 O(degree(u)) : u에서 v가 연결됐는지 보려면 u의 리스트를 다 뒤져야 합니다.
  - 공간 복잡도 O(V+E): 실제 존재하는 간선의 정보만 저장하므로 메모리 낭비가 적습니다.

그래프의 표현 방법

- 인접 리스트 (Adjacency List)

```
// 각 정점별로 연결된 노드를 담을 벡터 배열을 선언한다.
vector<int> adj[V + 1];

for (int i = 0; i < E; i++) {
    int u, v;
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u); // 무방향일 경우 양방향으로 연결 정보를 추가한다.
}
```

그래프의 표현 방법

비교 항목	인접 행렬 (Adjacency Matrix)	인접 리스트 (Adjacency List)
공간 복잡도	$O(V^2)$ (무조건 정점의 제곱)	$O(V+E)$ (실제 간선 수에 비례)
연결 확인 $(u \to v)$	$O(1)$ (즉시 확인 가능)	$O(degree(u))$ (리스트를 뒤져야 함)
이웃 노드 찾기	$O(V)$ (비어있는 칸도 다 확인)	$O(degree(u))$ (연결된 애들만 확인)
메모리 한계 int 4byte 기준	**V=10,000일 때:** 약 **400MB** (메모리 초과 위험) **V=100,000일 때:** 약 **40GB** (절대 불가)	**V=100,000, E=100,000일 때:** 실제 데이터 약 **800KB** + 벡터 오버헤드 → **수 MB 내외**로 매우 안전함
적합한 상황	1. 간선이 빽빽할 때 $(E \approx V^2)$ 2. 연결 여부를 $O(1)$로 계속 물어볼 때	1. 간선이 듬성듬성할 때 $(E \ll V^2)$ 2. 연결된 이웃 노드만 순차적으로 탐색할 때

그래프 순회 (Graph Traversal)

- 정의: 그래프의 모든 정점을 빠짐없이 딱 한 번씩 방문하는 과정입니다.
- 목적: 특정 정점을 찾거나,그래프의 구조(연결성, 사이클 등)를 파악하기 위해서 입니다.
- 필수 요소: 방문 여부 확인
  - 그래프에 사이클이 존재할 경우, 프로그램의 무한 루프를 방지합니다.
  - 이미 확인한 정점을 다시 방문하지 않음으로써 불필요한 중복 연산을 차단합니다.

방문 체크 배열 (visited[])

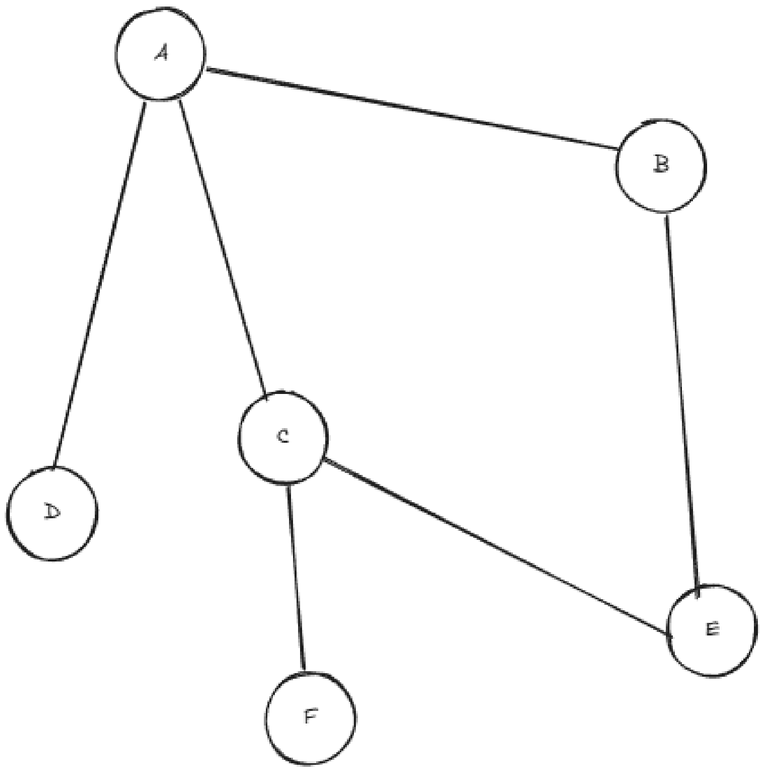
```
/*
 방문 체크 배열을 활용한 기본적인 탐색 로직 예시.
 배열의 인덱스는 각 정점의 번호에 대응하며, 값은 방문 여부를 나타낸다.
 */
bool visited[MAX_V];

void traverse(int curr) {
    // 1. 현재 노드 방문 처리
    visited[curr] = true;

    for (int next : adj[curr]) {
        // 2. 다음 노드가 아직 방문하지 않은 노드인지 확인
        if (!visited[next]) {
            // 3. 조건 만족 시에만 다음 탐색 진행
            traverse(next);
        }
    }
}
```

- 여기서는 배열을 사용하여 구현했지만, 상황에 맞게 다른 자료구조를 사용할 수 있습니다.
  - 예) 정점 번호가 크고 띄엄띄엄 있거나 정점의 이름이 문자열일때, set, unordered_set이 대안이 될 수 있습니다.

너비 우선 탐색 (BFS, Breadth-First Search)

- 개념: 가까운 곳부터 먼저 방문하고, 멀리 있는 곳은 나중에 간다.
- 작동 메커니즘
  - 시작 정점으로부터 거리가 1인 정점들을 모두 방문한 뒤, 거리가 2인 정점들을 방문하는 식으로 탐색을 진행합니다.
  - 최단 거리 탐색: 모든 간선의 가중치가 동일할 때, 처음 발견한 경로가 곧 최단 경로임이 보장됩니다.
- 비유
  - 파동 확산: 잔잔한 호수에 돌을 던졌을 때 퍼져나가는 동심원 모양의 물결입니다.

BFS의 방문 순서



방문 순서
A -> B -> C -> D -> E -> F

BFS의 구현 (큐)

```
// 큐를 활용하여 시작 정점으로부터 가까운 노드 순으로 탐색을 수행합니다.

void bfs(int start) {
    queue<int> q;
    q.push(start);
    visited[start] = true;

    while (!q.empty()) {
        int curr = q.front();
        q.pop();
        cout << curr << " ";

        for (const int& next : adj[curr]) {
            if (!visited[next]) {
                // 큐에 넣는 시점에 방문 처리하여 중복된 정점이 큐에 쌓이는 것을 방지
                visited[next] = true;
                q.push(next);
            }
        }
    }
}
```

BFS (큐) 특징

- 시간 복잡도 $O(V + E)$
  - 모든 정점과 간선을 딱 한 번씩만 확인하므로 매우 효율적입니다. (인접 리스트 기준)
- 공간 복잡도 O(V)
  - std::queue는 힙(Heap)영역을 사용합니다.
  - 최악의 경우(시작 노드에 모든 노드가 직접 연결된 경우 등) 모든 정점이 큐에 쌓일 수 있습니다.
- 탐색 경로 및 순서
  - 시작 노드에서 가까운 노드(거리순)부터 차례대로 방문하게 됩니다.
  - 결과적으로 층별 탐색(Level-order Traversal)이 이루어집니다.

특정 거리의 도시 찾기 BOJ 18352

- 문제
  - 1번부터 N번까지의 도시와 M개의 단방향 도로(간선)가 존재합니다.
  - 특정 도시 (출발지)로부터 출발하여 도착할 수 있는 모든 도시 중에서, 최단 거리가 정확히 K인 모든 도시들의 번호를 오름차순으로 출력해야 합니다.

특정 거리의 도시 찾기 BOJ 18352

- 문제
  - 1번부터 N번까지의 도시와 M개의 단방향 도로(간선)가 존재합니다.
  - 특정 도시 (출발지)로부터 출발하여 도착할 수 있는 모든 도시 중에서, 최단 거리가 정확히 K인 모든 도시들의 번호를 오름차순으로 출력해야 합니다.
- 해설
  - 도시의 개수 N이 최대 30만, 반드시 연결된 도로만 저장하는 인접 리스트를 사용해 그래프를 표현해야 합니다.
    ※ 인접 행렬을 쓰면 메모리(약 360GB)초과
  - visited 배열 대신, 거리를 저장하는 배열을 -1(미방문)로 초기화하여 사용합니다.
    ※ 방문 체크와 최단 거리를 동시에 처리할 수 있음
  - 아직 방문하지 않은 도시라면 (현재 도시의 거리 + 1)을 기록하고 큐에 넣습니다.

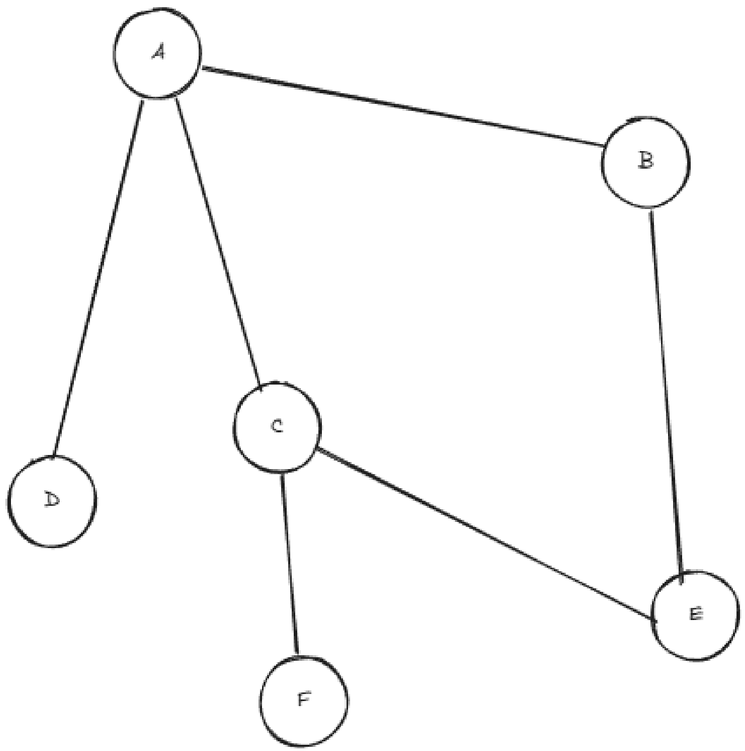
특정 거리의 도시 찾기 BOJ 18352

```
vector<vector<int>> adj(N + 1);
vector<int> dist(N + 1, -1);

for (int i = 0; i < M; i++) {
    int u, v;
    cin >> u >> v;
    adj[u].push_back(v);
}

queue<int> q;

// 시작점 설정
dist[X] = 0;
q.push(X);

while (!q.empty()) {
    int curr = q.front();
    q.pop();
    // 현재 도시에서 이동 가능한 모든 인접 도시를 확인한다.
    for (int next_node : adj[curr]) {
        // 아직 방문하지 않은 도시(-1)라면 최단 거리를 갱신하고 큐에 삽입한다.
        if (dist[next_node] == -1) {
            dist[next_node] = dist[curr] + 1;
            q.push(next_node);
        }
    }
}
```

깊이 우선 탐색 (DFS, Depth-First Search)

- 개념: 갈 수 있는 만큼 깊이 가고, 막히면 돌아온다.
- 작동 메커니즘
  - 현재 정점에서 연결된 미방문 정점이 있다면, 그 정점으로 이동하여 탐색을 이어갑니다.
  - 더 이상 갈 곳이 없는 막다른 길(Dead End)에 도달하면, 이전 정점으로 되돌아가서 탐색 가능한 경로를 찾습니다.
- 비유
  - 미로 찾기: 한 손으로 벽을 짚고 끝까지 가보다가 막히면 직전 갈림길로 돌아오는 전략입니다.

DFS의 방문 순서



방문 순서
A -> C -> E -> B -> F -> D

DFS의 구현 (재귀)

```
// 재귀 호출을 통해 깊이 우선 탐색을 수행한다.
// 인접 리스트를 순회하며 미방문 노드에 대해 자기 자신을 다시 호출한다.

vector<int> adj[10001];
bool visited[10001];

void dfs(int curr) {
    // 현재 정점을 방문 처리하여 재방문으로 인한 무한 루프를 차단한다.
    visited[curr] = true;
    cout << curr << " ";

    for (int next : adj[curr]) {
        if (!visited[next]) {
            dfs(next);
        }
    }
}
```

DFS의 특징 (재귀)

- 시간 복잡도 $O(V + E)$
  - 모든 정점과 간선을 딱 한 번씩만 확인하므로 매우 효율적입니다. (인접 리스트 기준)
- 공간 복잡도 O(V)
  - 최악의 경우(일자 그래프) 재귀 깊이가 정점 개수만큼 깊어질 수 있어 Stack Overflow에 유의해야 합니다.
- 탐색 경로 및 순서
  - 방문 순서: 가장 먼저 발견한 인접 노드를 따라 즉시 깊게 파고들며, 하나의 유효한 경로를 끝까지 추적합니다.
  - 함수 호출(진입) 시점에 즉시 방문 체크를 수행하므로, 우리가 머릿속으로 그리는 DFS와 완벽히 일치합니다.
  - 첫 번째 경로 탐색이 완전히 끝난 후에 바로 이전 갈림길(다음 인접 노드)의 탐색을 시작합니다.

DFS의 구현 (스택)

```
// STL stack을 사용하여 깊이 우선 탐색을 수행한다
void dfs(int start) {
    stack<int> s;
    s.push(start);

    while (!s.empty()) {
        int curr = s.top();
        s.pop();

        // 동일한 노드가 여러 경로를 통해 스택에 중복 삽입되었을 경우, 가장 깊은 경로만 인정
        if (visited[curr]) continue;

        visited[curr] = true;
        cout << curr << " ";
        // 스택의 LIFO 특성을 상쇄하고 정방향 탐색 우선순위를 유지하기 위해 역순 삽입
        for (int i = adj[curr].size() - 1; i >= 0; i--) {
            int next = adj[curr][i];
            if (!visited[next]) {
                s.push(next);
            }
        }
    }
}
```

DFS의 특징 (스택)

- 시간 복잡도 $O(V + E)$
  - 스택에 중복 삽입될 수는 있으나, 실제 방문 처리와 간선 확인은 한 번씩만 이루어지므로 O(V+E)를 보장합니다.
- 공간 복잡도 O(V)
  - std::stack은 힙(Heap)영역을 사용하는데 시스템 스택보다 넉넉하기 때문에 대규모 그래프 탐색에서 더 안전합니다.

컴백홈 BOJ 1189

- 문제
  - R x C 맵에서 왼쪽 아래(출발지)에서 오른쪽 위(도착지)로 이동하는 문제입니다.
  - 맵에 표시된 T는 장애물로 지나갈 수 없습니다.
  - 단순한 목적지 도달이 아닌, 이동 거리가 정확히 K인 경로의 총 개수를 구해야 합니다.

컴백홈 BOJ 1189

- 문제
  - R x C 맵에서 왼쪽 아래(출발지)에서 오른쪽 위(도착지)로 이동하는 문제입니다.
  - 맵에 표시된 T는 장애물로 지나갈 수 없습니다.
  - 단순한 목적지 도달이 아닌, 이동 거리가 정확히 K인 경로의 총 개수를 구해야 합니다.
- 풀이
  - 이 문제에서는 2차원 배열을 상하좌우로 1칸씩 연결되어 있는 그래프로 생각합니다.

    ※ 한 번에 더 멀리 이동할 수 있는 문제의 경우는 n칸씩 연결되어 있는 그래프로 생각
  - dx, dy 배열을 활용하여 현재 위치에서 상하좌우 4방향으로 다음 이동할 좌표를 계산합니다.
  - 해당 경로의 탐색을 마치고 이전 정점으로 돌아올 때는, 반드시 visited = False로 도장을 다시 지워야 합니다.

    ※ 그래야 전혀 다른 방향에서 출발한 우회 경로가 나중에 이 칸을 새로운 길의 일부로 정상적으로 밟고 지나갈 수 있음

컴백홈 BOJ 1189

```
// 상하좌우 탐색을 위한 방향 벡터
int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};

void dfs(int x, int y, int dist) {
    // 1. 기저 조건: 목적지(오른쪽 위)에 도착했을 때
    if (x == 0 && y == C - 1) {
        if (dist == K) answer++;
        return;
    }

    // 2. 4방향 탐색
    for (int i = 0; i < 4; i++) {
        int nx = x + dx[i];
        int ny = y + dy[i];

        // 맵 범위 확인 및 장애물('T')/방문 여부 체크
        if (nx >= 0 && nx < R && ny >= 0 && ny < C) {
            if (grid[nx][ny] != 'T' && !visited[nx][ny]) {

                visited[nx][ny] = true;
                dfs(nx, ny, dist + 1);
                // 다른 경로에서 이 칸을 다시 사용할 수 있도록 함수 종료 시 도장을 지워준다.
                visited[nx][ny] = false;
            }
        }
    }
}
```

그 외 연습 문제

- 11724 연결 요소의 개수
- 2606 바이러스
- 7596 토마토
- 1697 숨바꼭질